

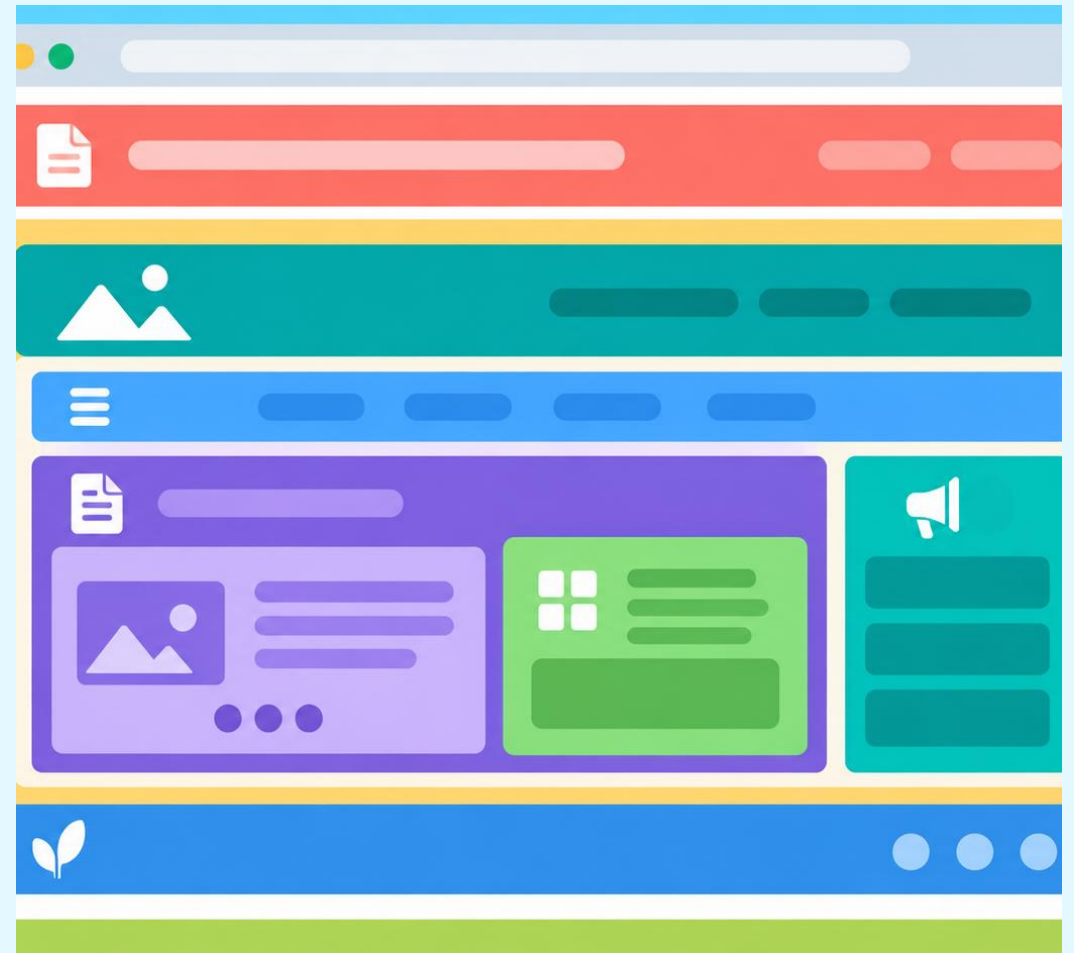
A web működése és a HTML alapjai

Kliens–szerver gondolkodás, HTML dokumentum, elemek, attribútumok, szemantika és helyes struktúra



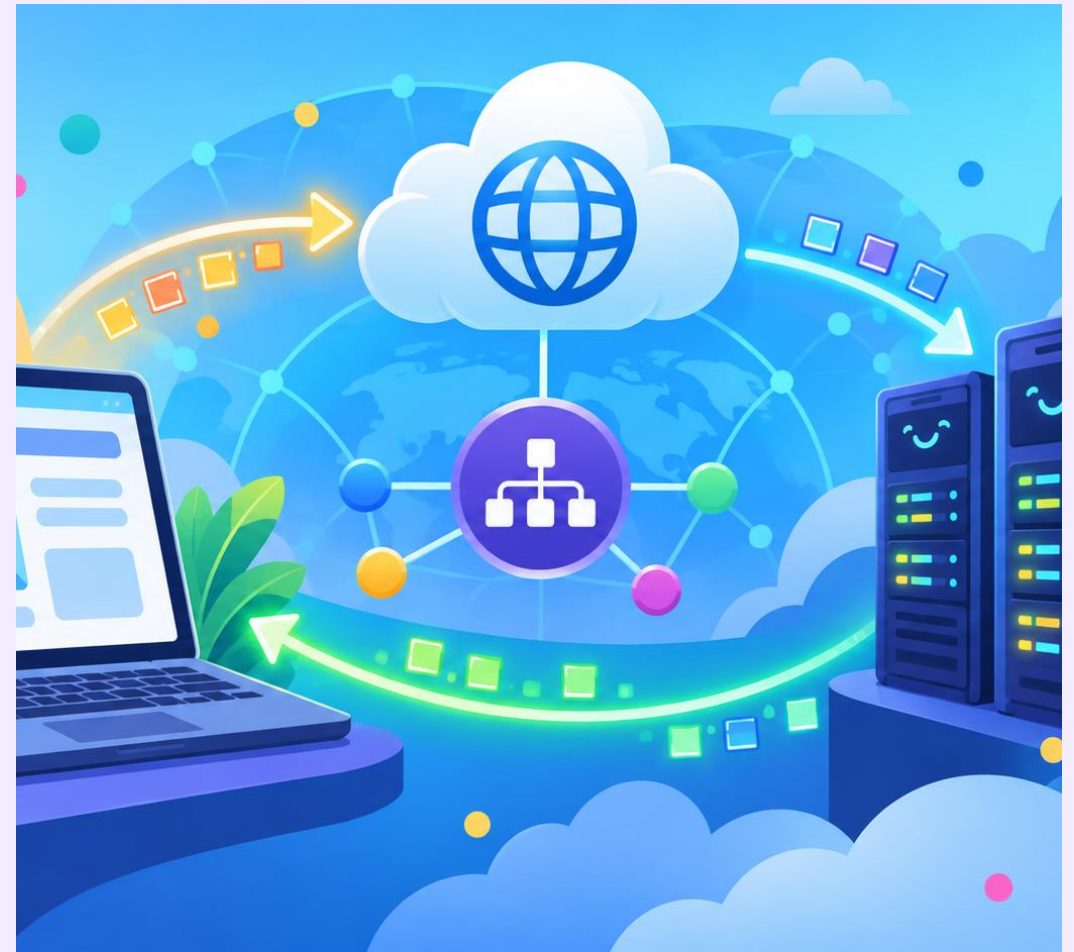
Miről lesz szó?

Mi történik, amikor beírunk egy webcímet?
Mi a kliens és mi a szerver feladata?
Mit jelent a kérés és válasz a weben?
Miből épül fel egy HTML dokumentum?
Miért fontosak a szemantikus elemek?



A web röviden: dokumentumok hálózata

A web nem azonos magával az internettel: az internet az infrastruktúra, a web erre épülő szolgáltatás.
A böngésző weboldalakat kér le, majd megjeleníti azokat a felhasználónak.
A weboldal alapja gyakran HTML, CSS és JavaScript együttese.
A linkek összekötik a dokumentumokat, ezért beszélünk „hálóról”.



A szereplők: kliens, hálózat, szerver

Kliens: a felhasználó eszköze és böngészője.

Hálózat: az útvonal, amelyen a kérés és válasz halad.

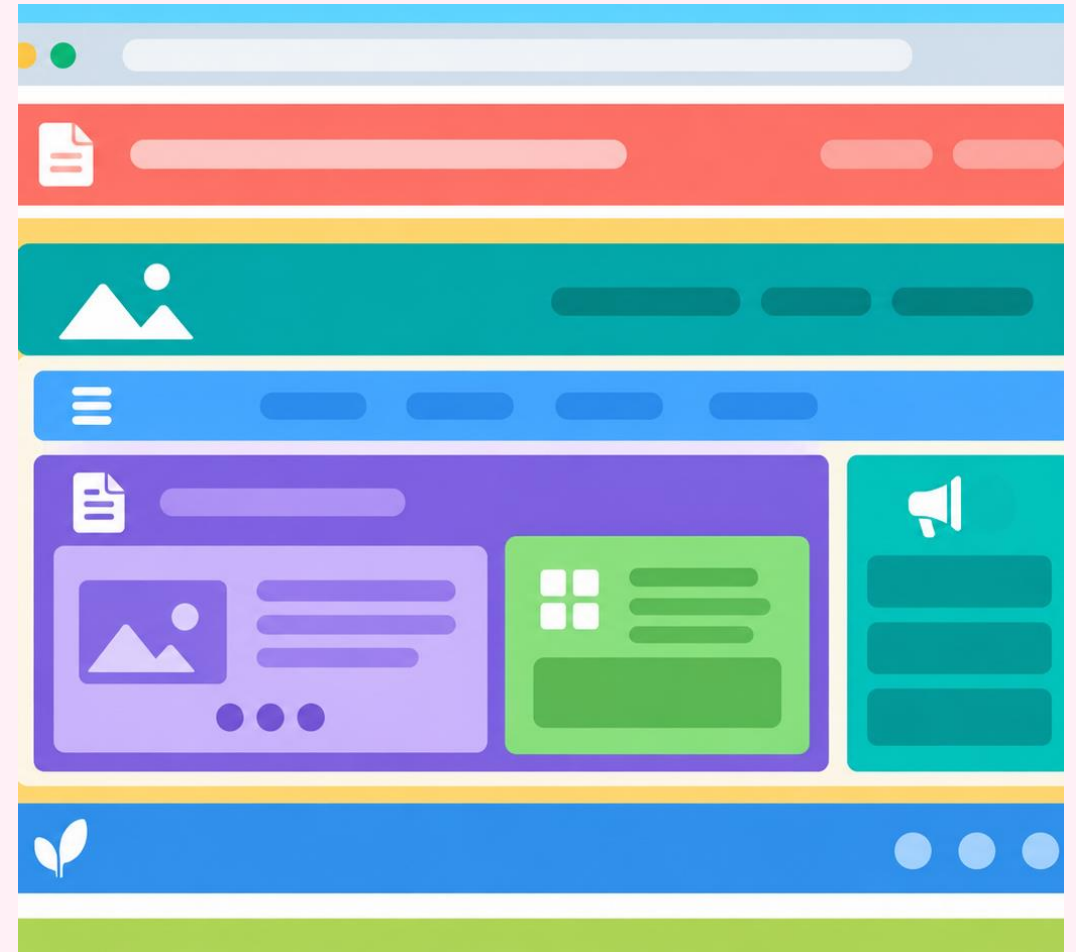
Szerver: távoli gép vagy szolgáltatás, amely erőforrást ad vissza.

Adatbázis: sok webalkalmazás mögött innen érkezik a tartalom.



A kliens feladata

URL beírása vagy link megnyitása.
HTTP kérés elküldése a megfelelő szerver felé.
A kapott HTML, CSS, JS és képek letöltése.
A HTML feldolgozása DOM-má.
A kész oldal kirajzolása és interaktív kezelése.

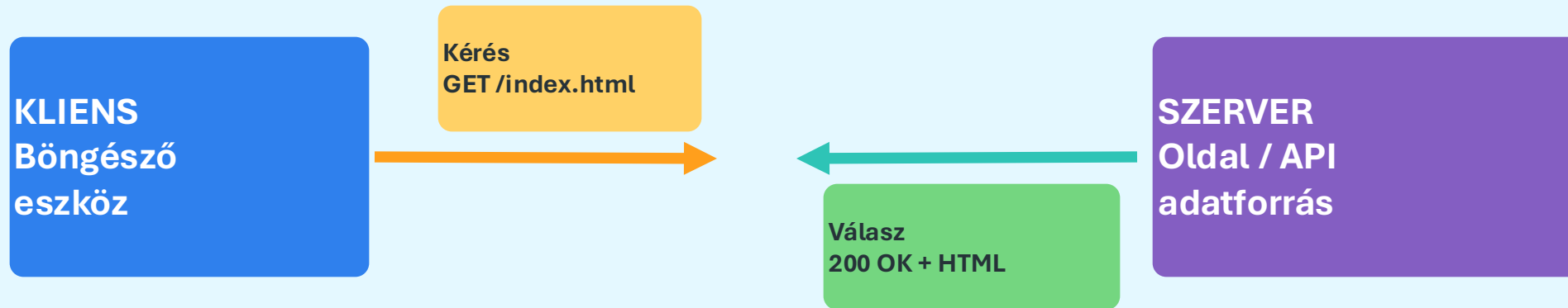


A szerver feladata

Fogadja és értelmezi a beérkező kéréseket.
Eldönti, milyen erőforrásra van szükség: HTML fájl, kép, API
válasz vagy generált oldal.
Szükség esetén adatbázisból kér adatot.
HTTP választ küld státuszkóddal, fejlécekkel és tartalommal.



Kérés és válasz: a web beszélgetése



A kliens kérdez: „kérem ezt az oldalt”.

A szerver válaszol: „itt a tartalom, így kezeld”.

A válasz lehet sikeres, átirányított vagy hibás.

A böngésző sok kérést indíthat egyetlen oldalhoz: HTML, CSS, JS, képek, fontok.

URL: a webcím részei

https://
protokoll

pelda.hu
domain

/termek/cipó
útvonal

?szin=piros
query

#meretek
fragment

Protokoll: például https.

Domain: a szerver könnyen megjegyezhető neve.

Útvonal: megmutatja, melyik erőforrást kérjük.

Query paraméter: plusz adat a kéréshez.

Fragment: oldalrészben belüli hivatkozás.

DNS: név helyett IP-cím

A felhasználó domainnevet ír be, például egy webhely nevét.
A DNS megkeresi, milyen IP-cím tartozik a domainhez.
A böngésző ezután már az IP-cím felé tud hálózati kapcsolatot nyitni.
A folyamat általában automatikus és nagyon gyors.



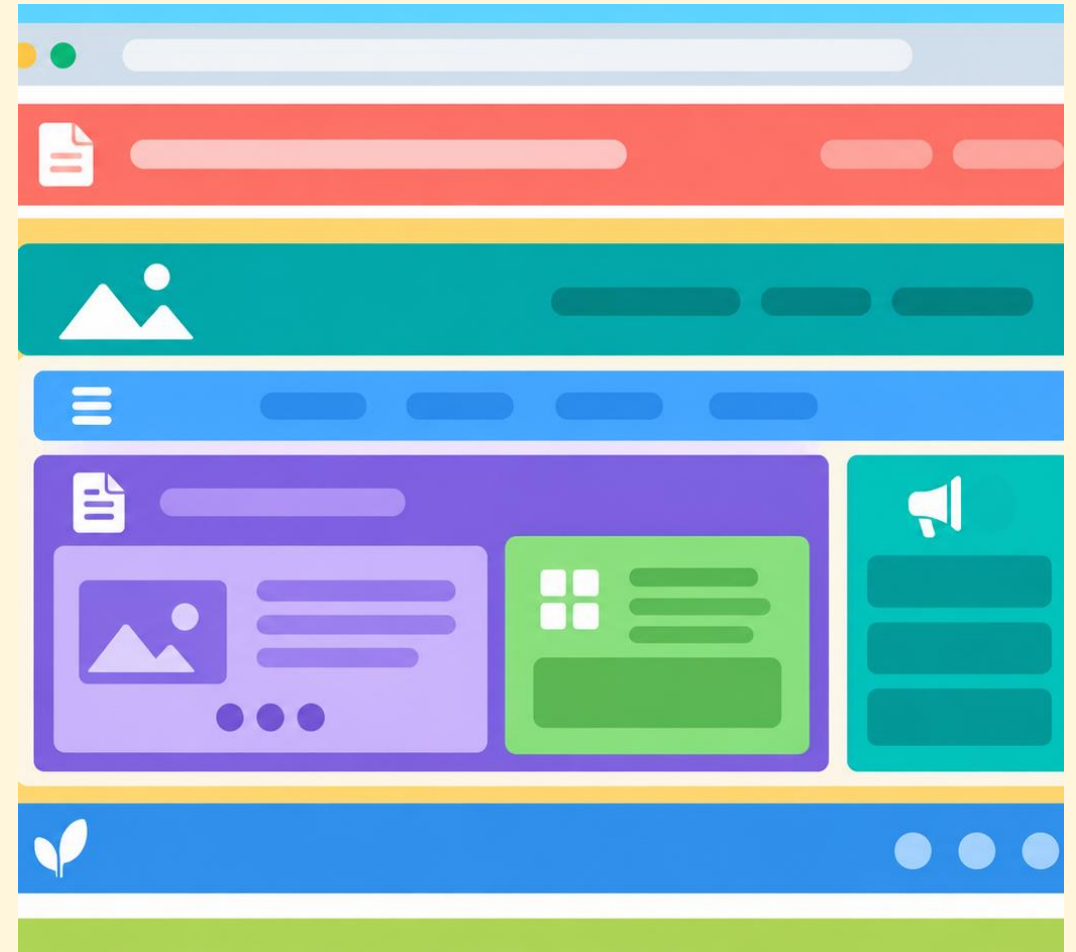
HTTPS: biztonságos kapcsolat

A HTTPS titkosított kapcsolatot jelent a böngésző és a szerver között.
Segít megakadályozni, hogy útközben olvassák vagy módosítsák az adatokat.
Űrlapoknál, bejelentkezésnél és fizetésnél alapvető.
A lakat ikon nem azt jelenti, hogy a webhely biztosan megbízható, csak azt, hogy a kapcsolat titkosított.



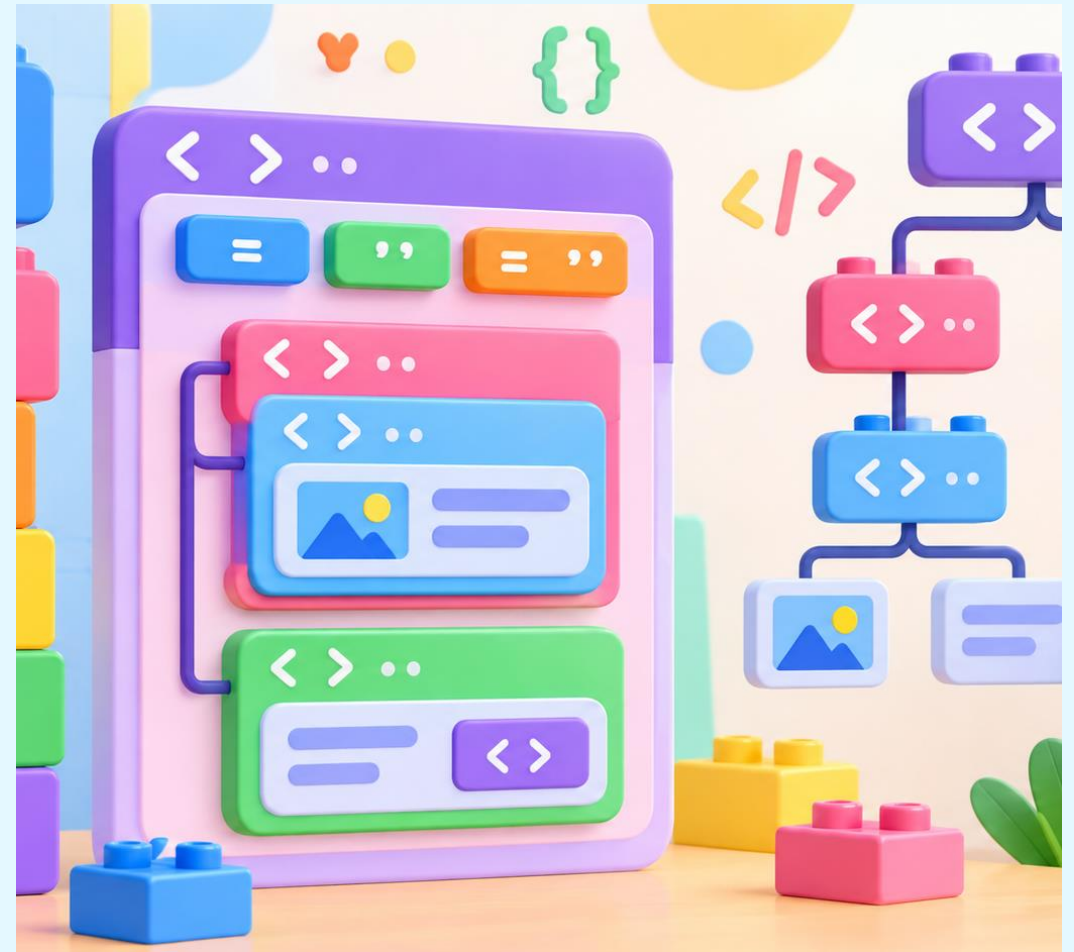
Mit kap vissza a böngésző?

HTML: a dokumentum szerkezete és tartalma.
CSS: a megjelenés, elrendezés, színek, betűk.
JavaScript: viselkedés, interakció, dinamikus működés.
Képek, videók, fontok és egyéb fájlok: kiegészítő erőforrások.



HTML: a dokumentum váza

A HTML nem programozási nyelv, hanem leíró jelölőnyelv.
Elemekkel írjuk le, hogy mi van az oldalon.
A böngésző a HTML alapján építi fel a dokumentum belső fa-szerkezetét.
A jó HTML logikus, olvasható és gépek számára is értelmezhető.



A minimális HTML dokumentum

```
<!doctype html>
<html lang="hu">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Oldal címe</title>
  </head>
  <body>
    <h1>Főcím</h1>
    <p>Az oldal tartalma.</p>
  </body>
</html>
```

<!doctype html>: modern HTML dokumentum jelzése.

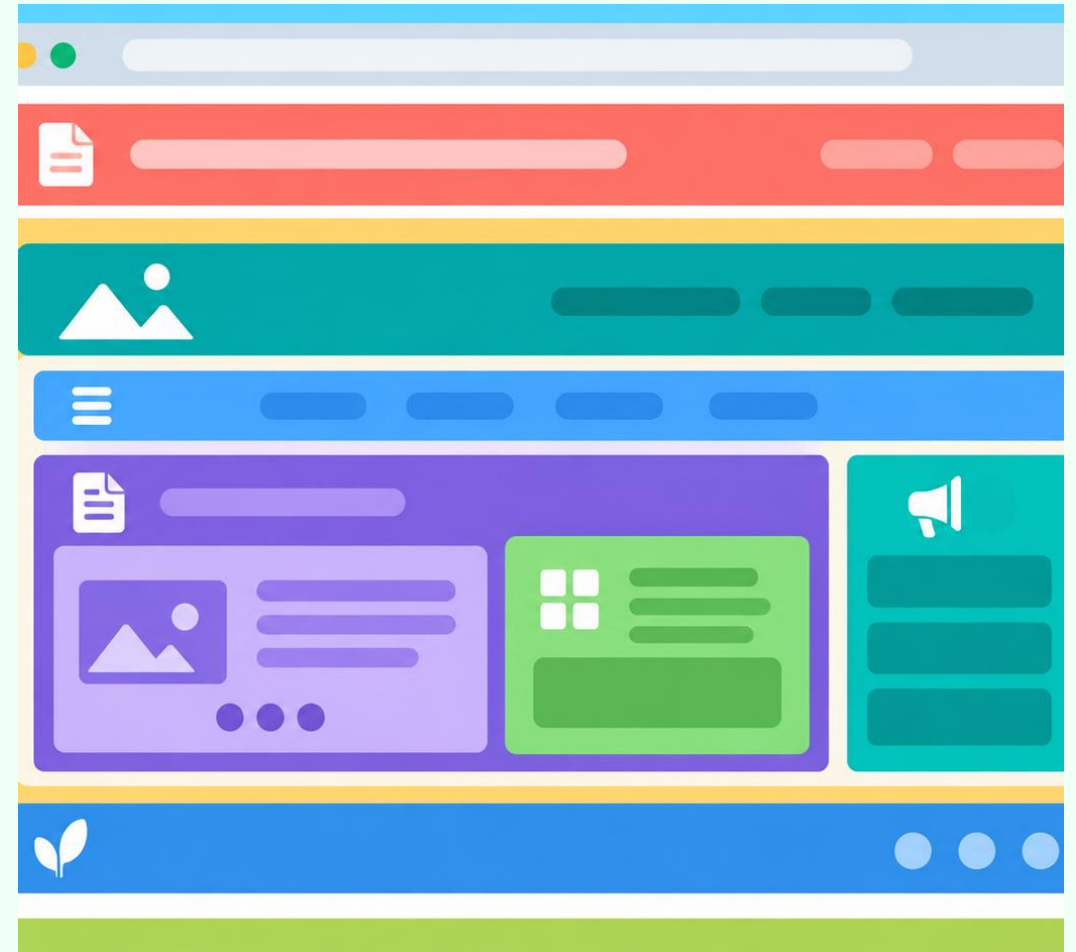
<html lang="hu">: a dokumentum gyökere és nyelve.

<head>: metaadatok, cím, kapcsolt fájlok.

<body>: amit a felhasználó az oldalon lát.

A head és a body különbsége

A head nem az oldal látható tartalmának helye.
Ide kerül a title, a karakterkódolás, viewport beállítás, CSS hivatkozás és metaadatok.
A body tartalmazza a címsorokat, bekezdéseket, képeket, navigációt, cikkeket és űrlapokat.
A kettő szétválasztása tiszta dokumentumot eredményez.



Alapvető HTML elemek

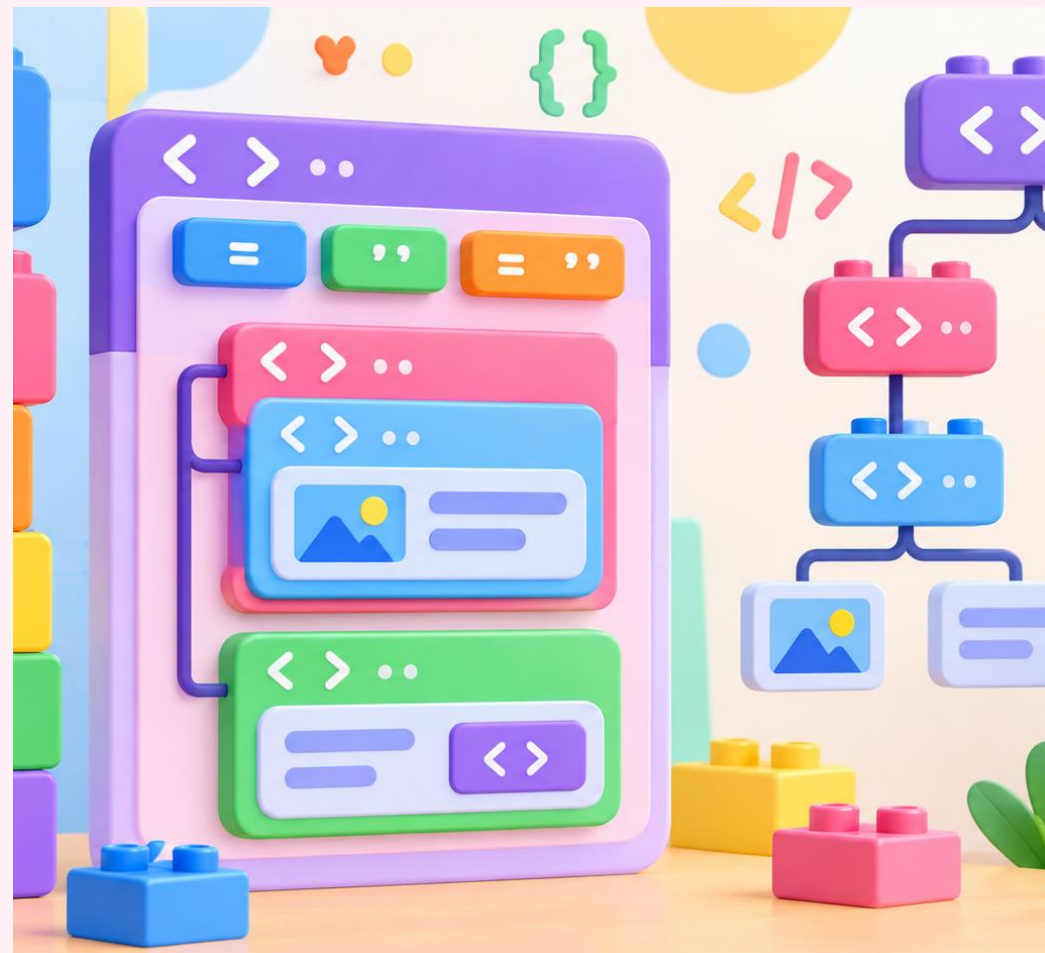
Címsorok: h1–h6, logikai hierarchiához.

Bekezdés: p, hosszabb szöveges tartalomhoz.

Link: a, másik oldalra vagy oldalrészre mutat.

Kép: img, kötelezően hasznos alt szöveggel.

Lista: ul, ol, li, felsorolásokhoz és lépésekhez.



Attribútumok: extra információk az elemeken

```
<a href="/kapcsolat">Kapcsolat</a>  
  
<section id="hirek" class="doboz kiemelt">  
...  
</section>
```

Az attribútum az elem nyitó tagjében szerepel.

Például href a link célját, src a kép forrását adja meg.

Az id egyedi azonosító, a class csoportosítás és CSS célzás miatt fontos.

Az alt nem dísz: képeknél akadálymentességi és tartalmi jelentősége van.

Blokk és inline gondolkodás

Blokkszintű elemek általában új sorban jelennek meg és szerkezeti egységet alkotnak.

Inline elemek a szöveg folyásában maradnak.

A CSS sok mindent megváltoztathat, de a HTML jelentése ettől még fontos marad.

Ne azért válasszunk elemet, mert alapból jól néz ki, hanem mert jelentésben oda illik.

Példa a gondolkodásra

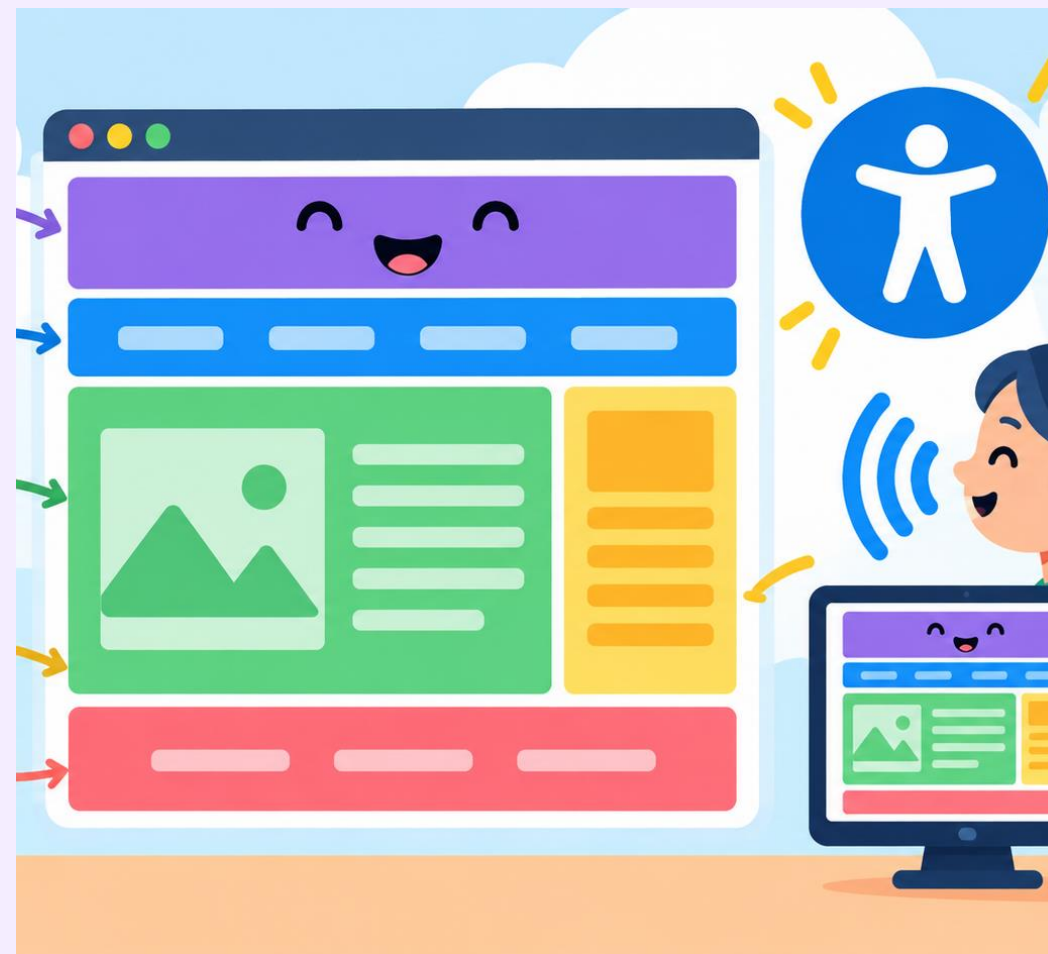
1. div = nagyobb doboz

2. span = szövegdarab

3. CSS nem jelentés

Szemantikus HTML: jelentést adunk

A szemantikus elem elárulja, milyen szerepe van a tartalomnak.
A header, nav, main, article, section, aside és footer többet mond, mint egy sima div.
A keresők, képernyőolvasók és fejlesztők is jobban értik az oldalt.
A szemantika nem díszítés, hanem szerkezeti és tartalmi információ.



Tipikus szemantikus elemek

header: bevezető rész vagy fejléc.

nav: fő navigációs linkek.

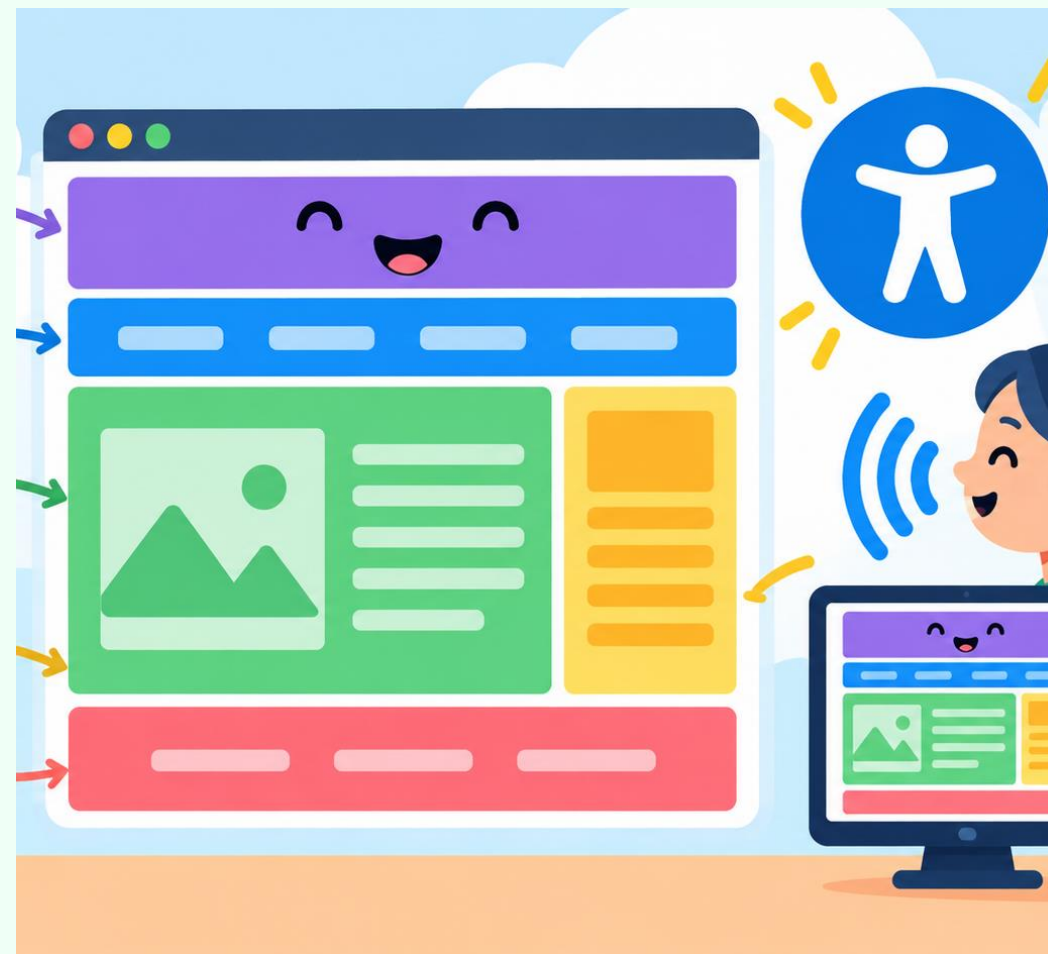
main: az oldal egyedi fő tartalma.

section: tematikus tartalmi egység.

article: önállóan is értelmezhető tartalom.

aside: kiegészítő vagy kapcsolódó tartalom.

footer: záró információk, jogi rész, elérhetőség.



Helyes címsor-hierarchia

Címsor-fa

1. h1: oldal címe

2. h2: fő fejezet

3. h3: alfejezet

Egy oldalnak legyen világos h1 címe.

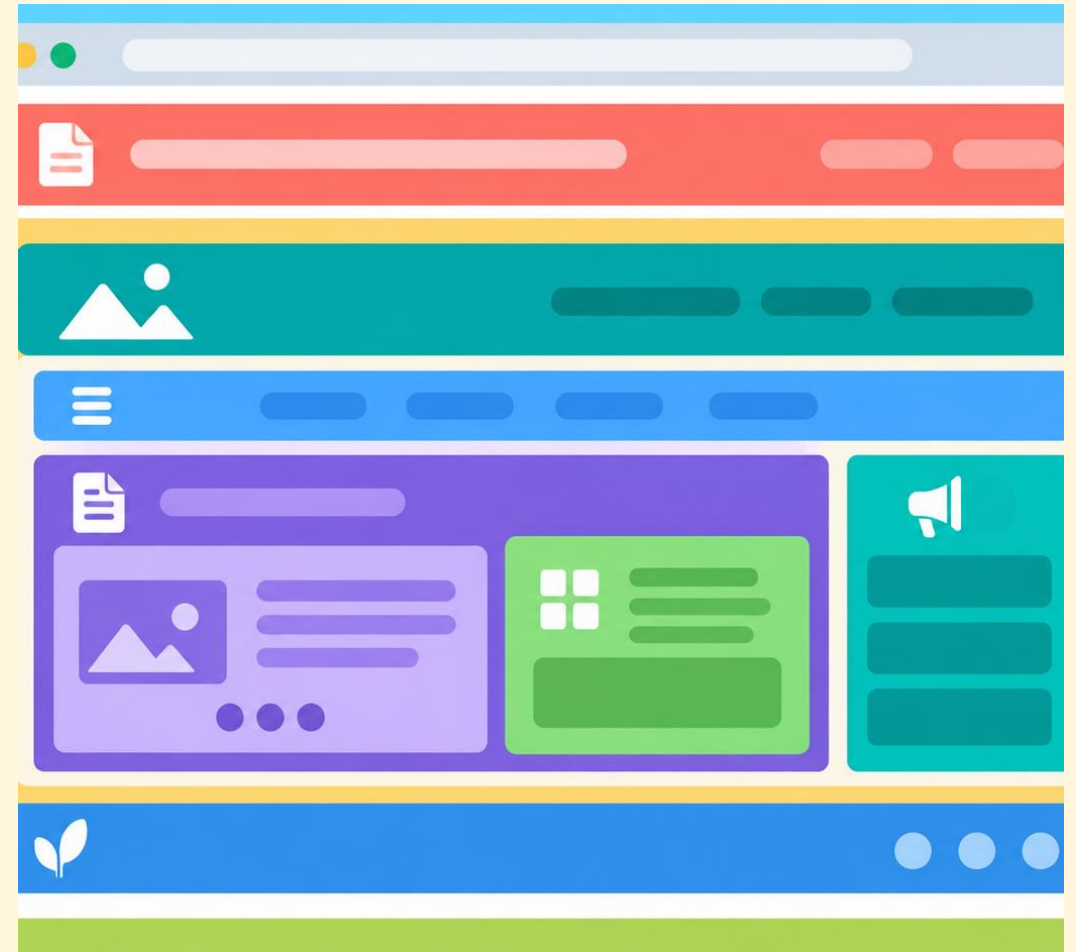
A h2, h3, h4 alárendelést jelöl, nem betűméretet.

Ne ugorjunk szinteket csak a kinézet miatt.

A címsorok alapján gyorsan átláthatóvá válik az oldal tartalma.

Dokumentumstruktúra: tiszta váz

Először gondolkodjunk tartalmi egységekben.
Utána válasszuk ki a megfelelő HTML elemeket.
A CSS csak ezután jöjjön, mert a kinézet nem pótolja a logikát.
A jó struktúra később könnyebben bővíthető, javítható és
újrahasznosítható.



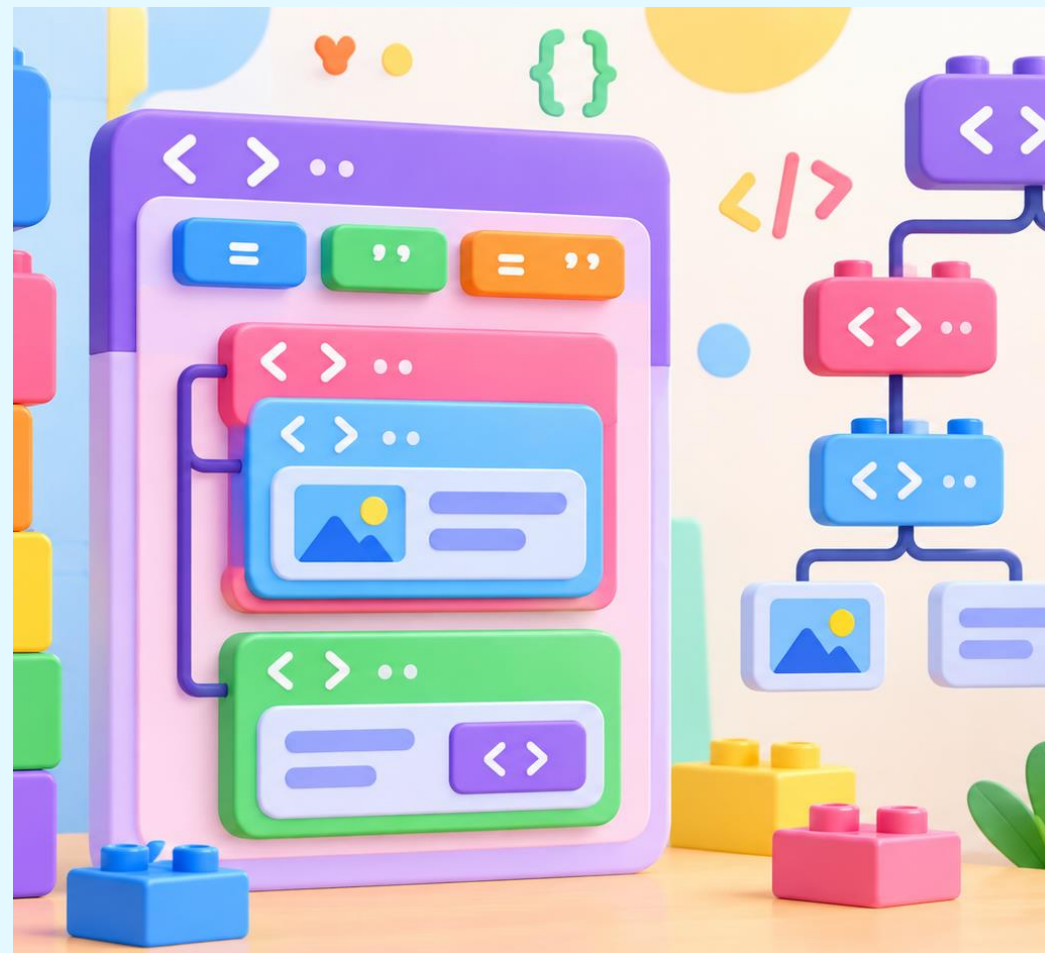
div és span: mikor használjuk?

A div általános blokk szintű tároló, ha nincs jobb szemantikus elem.

A span általános inline jelölő kisebb szövegrészekhez.

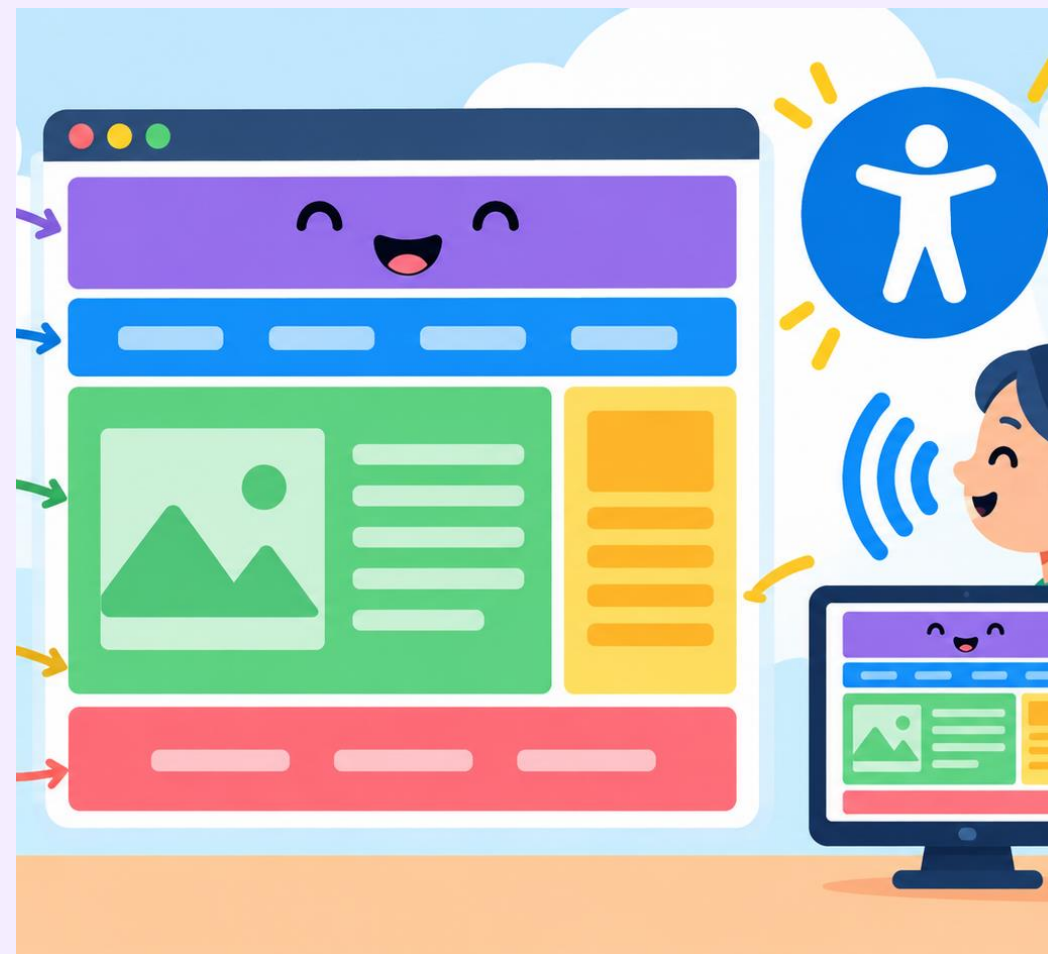
Nem hiba használni őket, de túlhasználva elveszik a dokumentum jelentése.

Előbb kérdezzük meg: létezik erre pontosabb HTML elem?



Akadálymentesség alapjai

A HTML legyen billentyűzettel is bejárható.
A képek alt szövege írja le a fontos információt.
A címkék és űrlapmezők kapcsolata legyen egyértelmű.
A szemantikus struktúra segíti a képernyőolvasókat.



Gyakori hibák kezdő HTML-ben

Mindenből div készül, jelentés nélkül.
Címsorokat csak látvány miatt választanak.
Hiányzik a lang, meta charset vagy viewport.
A képeknél nincs alt attribútum.
A nyitó és záró tagek összekeverednek.
A main, nav és footer szerepe nem különül el.

Javítási irány

1. validátor

2. alt szöveg

3. szemantika

4. rendezett behúzás

Mini példa: gondolkodjunk szerkezetben

Nem az a kérdés, hogy „hogyan legyen kék?”, hanem hogy „mi ez a tartalom?”.

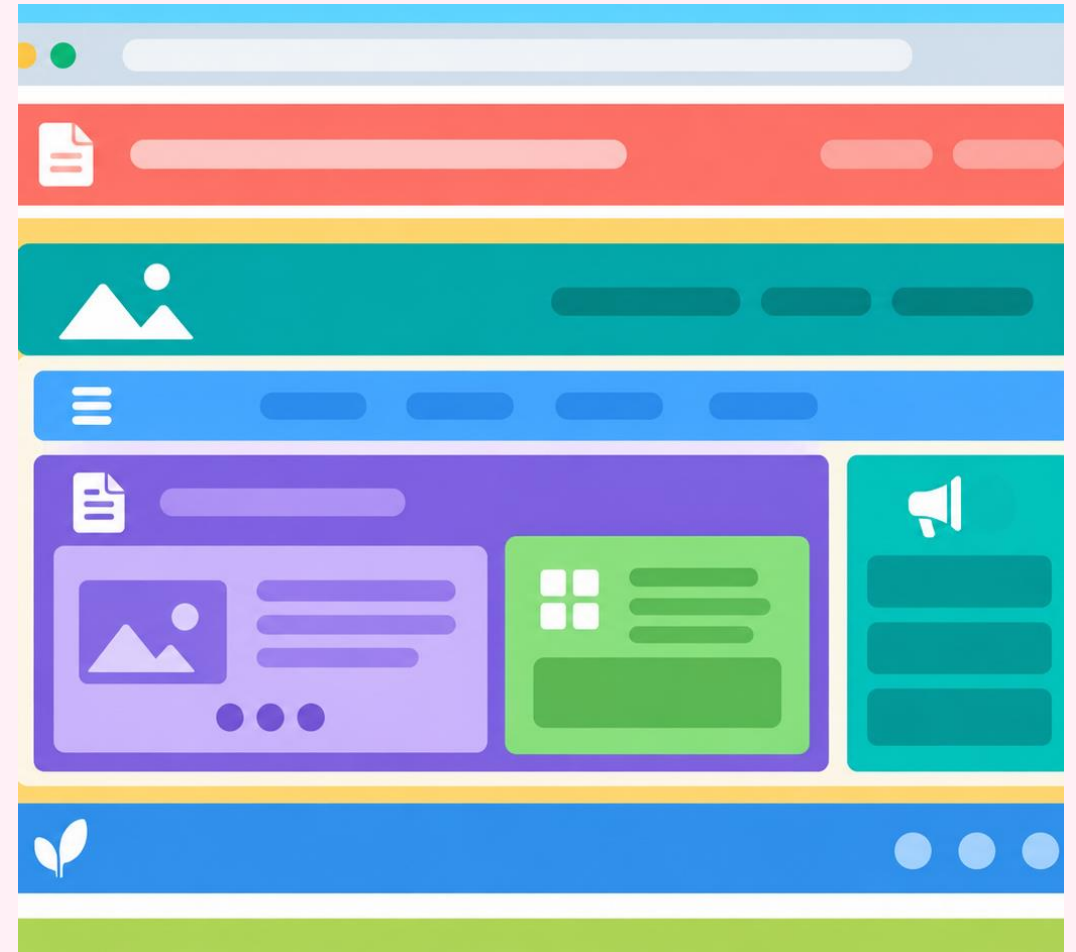
Főcím → h1.

Navigáció → nav.

Önálló bejegyzés → article.

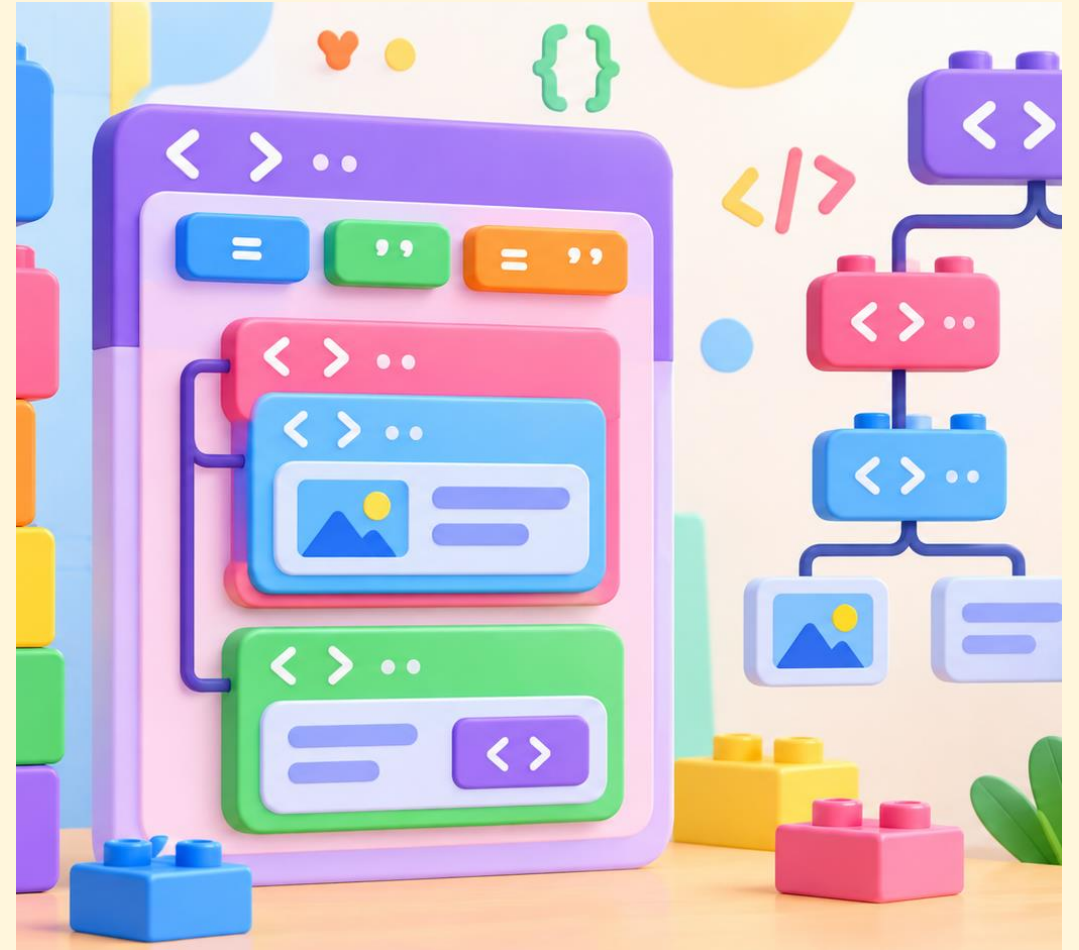
Kapcsolódó doboz → aside.

Záró információ → footer.



Ellenőrző kérdések

Mi történik először: HTML értelmezés vagy CSS alkalmazás?
Miért nem elég mindenre divet használni?
Mi a különbség a head és a body között?
Miért fontos az alt attribútum?
Mikor article és mikor section?



Összegzés

A web kliens és szerver közötti kérés-válasz folyamatokra épül.
A böngésző több erőforrásból rakja össze az oldalt.
A HTML a dokumentum jelentéssel bíró váza.
Az attribútumok pontosítják az elemek működését és jelentését.
A szemantikus, tiszta struktúra jobb olvashatóságot, hozzáférhetőséget és karbantarthatóságot ad.

